



testing_linter

Johan Alexander Peña Ortiz

Cristian Andrés Díaz Ortega

Juan Carlos Manrique Benavides

Martin José Botina Gonzalez

Oscar Leonardo Riveros Perez

Grupo GPT

Universidad Nacional de Colombia

Ambiente y Herramientas

Para la realización del análisis estático del código fuente se utilizaron las herramientas **Pylint** y **Pylint-Django**. La integración de **Pylint-Django** permitió el reconocimiento adecuado de los componentes propios del framework Django, reduciendo la aparición de advertencias incorrectas durante el proceso de análisis.

Con el fin de garantizar la compatibilidad con la estructura del proyecto y adaptar las validaciones a los criterios definidos para el desarrollo, se configuraron las directivas descritas en la **Tabla 1**.

Directiva	Valor configurado	Propósito
init-hook	<code>import os, sys sys.path.insert(0, os.getcwd())</code>	Agrega el directorio raíz del proyecto al path de Python.
load-plugins	<code>pylint_django</code>	Soporte para proyectos desarrollados con Django.
django-settings-module	<code>config.settings</code>	Especifica el módulo de configuración de Django.
missing-module-docstring	Deshabilitada	Evita advertencias por ausencia de documentación a nivel de módulo.
missing-class-docstring	Deshabilitada	Evita advertencias por ausencia de documentación en clases.
missing-function-docstring	Deshabilitada	Evita advertencias por ausencia de documentación en funciones y métodos.

Tabla 1: Reglas aplicadas al linter

Al ejecutar el comando `pylint .\information_app\` se obtiene el resultado que se observa en la **Figura 1**:

[illegible]

[illegible]

Figura 1: Recomendaciones del linter

Los errores y/o advertencias importantes se listan en la **Tabla 2** que se observa a continuación:

Prioridad	Tipo de Error	Código	Impacto	Recomendación
Alta	Importaciones con *	W0401	Contamina el namespace, dificulta el mantenimiento y genera otros warnings derivados.	Reemplazar <code>import *</code> por importaciones explícitas.
Alta	Captura genérica de excepciones	W0718	Oculto errores reales del sistema y dificulta el debugging.	Reemplazar <code>except Exception</code> por excepciones específicas (<code>ValueError</code> , <code>DatabaseError</code> , etc.).

Alta	Código duplicado	R0801	Incrementa la deuda técnica y el esfuerzo de mantenimiento.	Extraer la lógica repetida a funciones utilitarias compartidas.
Alta	Imports no utilizados	W0611 / W0614	Añaden ruido al código y dificultan su comprensión.	Eliminar importaciones que no se utilizan.
Media	Función con demasiadas ramas	R0912	Reduce la legibilidad y dificulta futuras modificaciones.	Refactorizar <code>update_equipment()</code> utilizando estructuras de mapeo o funciones auxiliares.
Media	Demasiados argumentos en una función	R0913 / R0917	Hace más difícil entender y mantener la interfaz del método.	Utilizar DTOs, <code>dataclass</code> o diccionarios para agrupar parámetros relacionados.
Media	Demasiados retornos	R0911	Incrementa la complejidad de lectura y mantenimiento.	Dividir la lógica en métodos más pequeños y reutilizables.
Baja	Líneas demasiado largas	C0301	Afecta únicamente la legibilidad.	Dividir expresiones largas en múltiples líneas.
Baja	Espacios en blanco al final de línea	C0303	Problema únicamente de estilo.	Configurar el editor para eliminarlos automáticamente.
Baja	Falta salto de línea al final del archivo	C0304	Problema menor de formato.	Agregar una línea vacía al final del archivo.
Ignorable	Advertencias en migraciones de Django	C0103, C0301, etc.	Son archivos autogenerados y normalmente no se modifican manualmente.	Excluir la carpeta <code>migrations</code> del análisis de pylint.

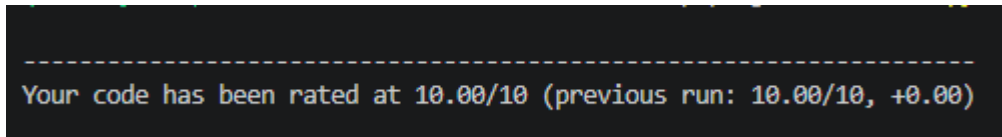
Tabla 2: Recomendaciones del Linter

Para poder abarcar todas las recomendaciones de Pylint se tuvo que reformar el código para evitar código redundante, aplicando las siguientes estrategias:

- **Modularización y Separación de Responsabilidades:** Se extrajeron módulos dedicados para la lógica de autenticación (`auth_services.py`) y se centralizaron funciones repetitivas en utilidades compartidas (`services_utils.py`, `repository_utils.py`), reduciendo significativamente el acoplamiento.

- **Abstracción y Principio DRY:** Se implementó una clase `BaseRepository` para estandarizar las operaciones de base de datos y se refactorizaron las vistas de depuración (`DebugView`) para que hereden de las vistas principales, eliminando la duplicación de código.
- **Limpieza y Estándares de Formato (PEP 8):** Se corrigió la longitud de las líneas, se organizaron los imports, se eliminaron variables no utilizadas y código inalcanzable, y se estandarizó la nomenclatura en todo el proyecto.
- **Manejo Robusto de Excepciones:** Se creó una jerarquía personalizada de excepciones (`AppException`) y se aplicó el encadenamiento de errores (`raise ... from exc`) para garantizar trazas legibles y un control de errores consistente.
- **Consistencia en el Acceso a Datos:** Se unificó el acceso a los repositorios dentro de los servicios (usando instancias propias `self.repository`) y se optimizaron las firmas de métodos mediante el uso de diccionarios consolidados.

Gracias a esto se logró un 10/10 según Pylint obteniéndose el resultado que se observa en la **Figura 2**.



```
-----  
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

Figura 2: Recomendaciones del linter aplicadas